# CMPE 281 – Assignment: Serverless CRUD API with DynamoDB

Name: Priya Shah | Student ID: 020176341

*Generated sample submission with dummy data, tables, and figures for PDF extraction evaluation.*

This sample write-up demonstrates a Lambda function invoked by API Gateway to perform CRUD operations against a DynamoDB table. The report intentionally mirrors a classroom submission format and uses entirely dummy request/response data.

## 1. Services Used

- AWS Lambda: Executes Python code without managing servers and scales automatically for intermittent traffic.
- Amazon DynamoDB: Stores student records in a key-value table with a string partition key named student_id.
- Amazon API Gateway: Exposes REST endpoints and forwards structured events to the Lambda handler.

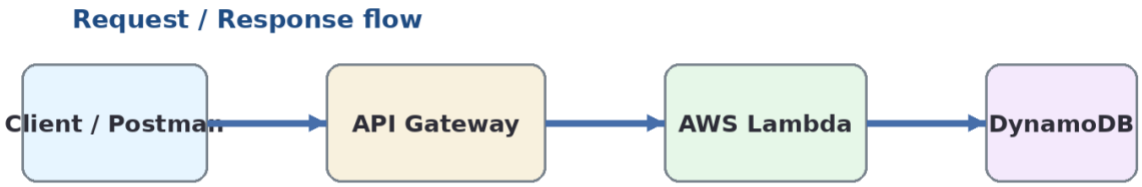


*Figure 1. Simplified request path from client to database.*

## 2. Lambda Handler

```
import json
import boto3

dynamodb = boto3.resource('dynamodb')
student_table = dynamodb.Table('StudentRecords')

def lambda_handler(event, context):
    method = event['httpMethod']
```

```
    if method == 'POST':
        student = json.loads(event['body'])
        student_table.put_item(Item=student)
        return {'statusCode': 200, 'body': json.dumps('Student added')}

    if method == 'GET':
        sid = event['queryStringParameters']['student_id']
        response = student_table.get_item(Key={'student_id': sid})
        if 'Item' not in response:
            return {'statusCode': 404, 'body': json.dumps('Record not found')}
        return {'statusCode': 200, 'body': json.dumps(response['Item'])}

    return {'statusCode': 405, 'body': json.dumps('Method not allowed')}
```

The remaining PUT and DELETE branches follow the same event shape, updating or deleting the record after a key lookup.

## 3. API Test Screenshots

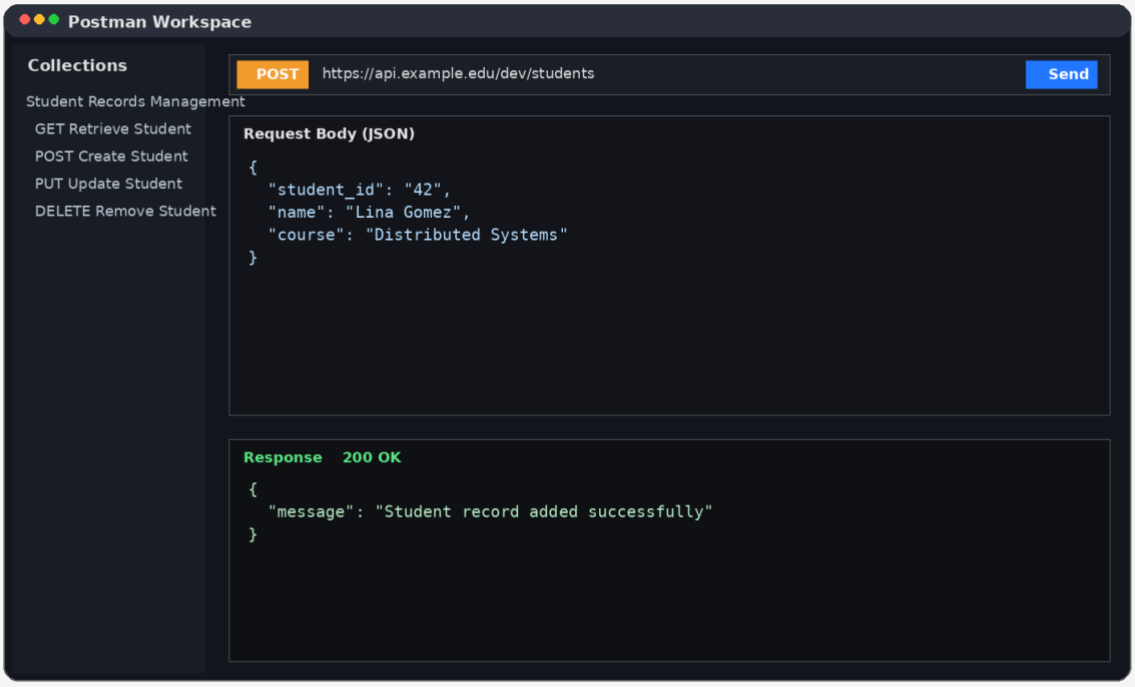


*Figure 2. Sample POST request used to add a new record.*

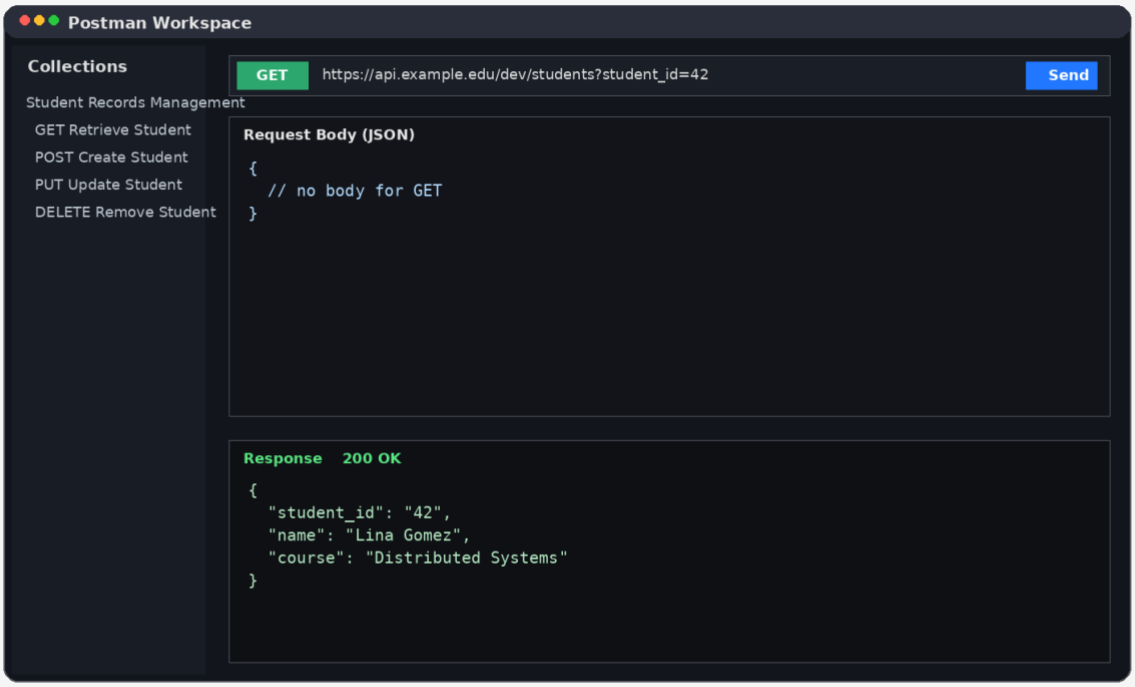


*Figure 3. Sample GET request retrieving the stored record.*

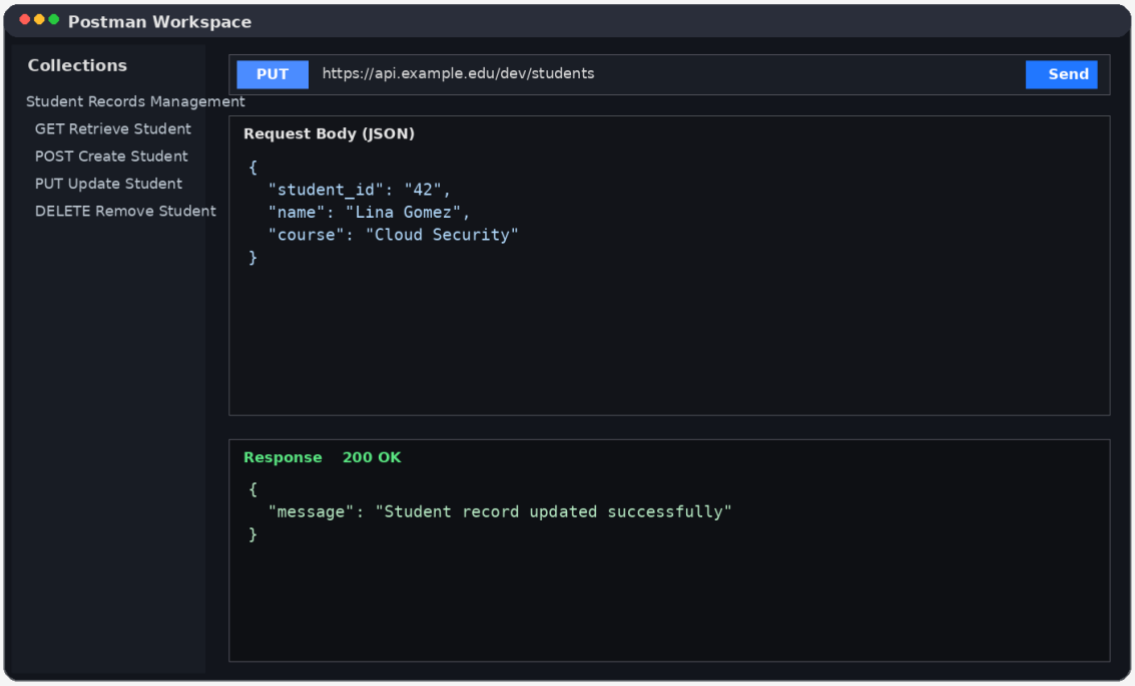


*Figure 4. PUT request used to update the course field.*

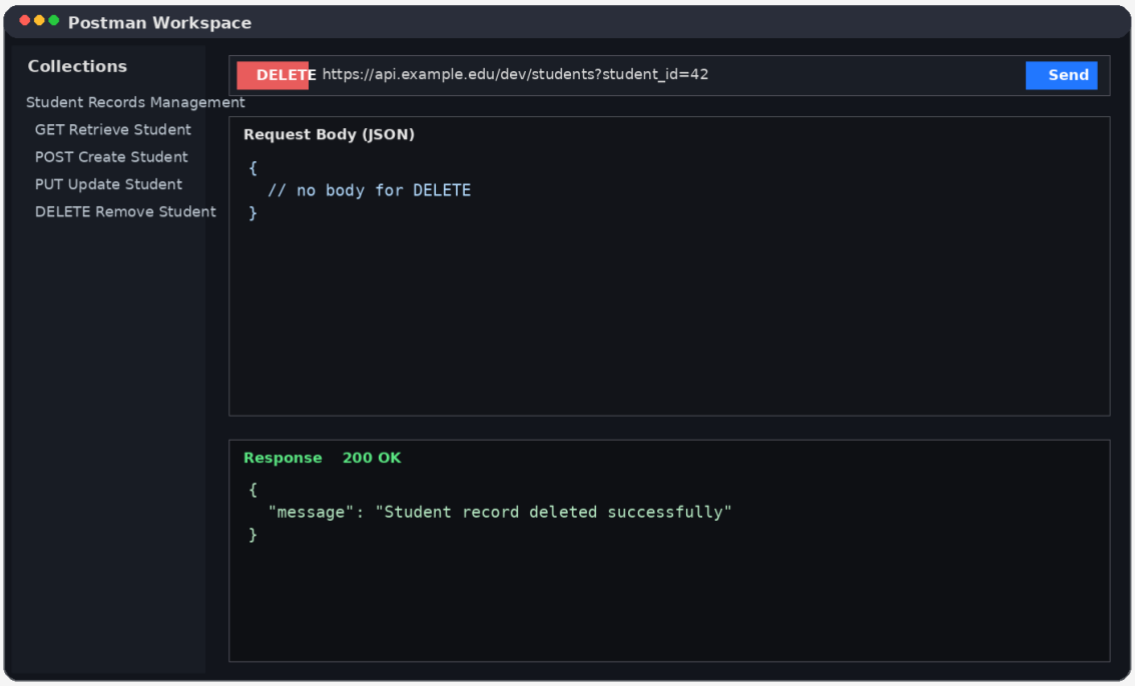


*Figure 5. DELETE request for removing the same record.*

## 4. DynamoDB State and Functional Results

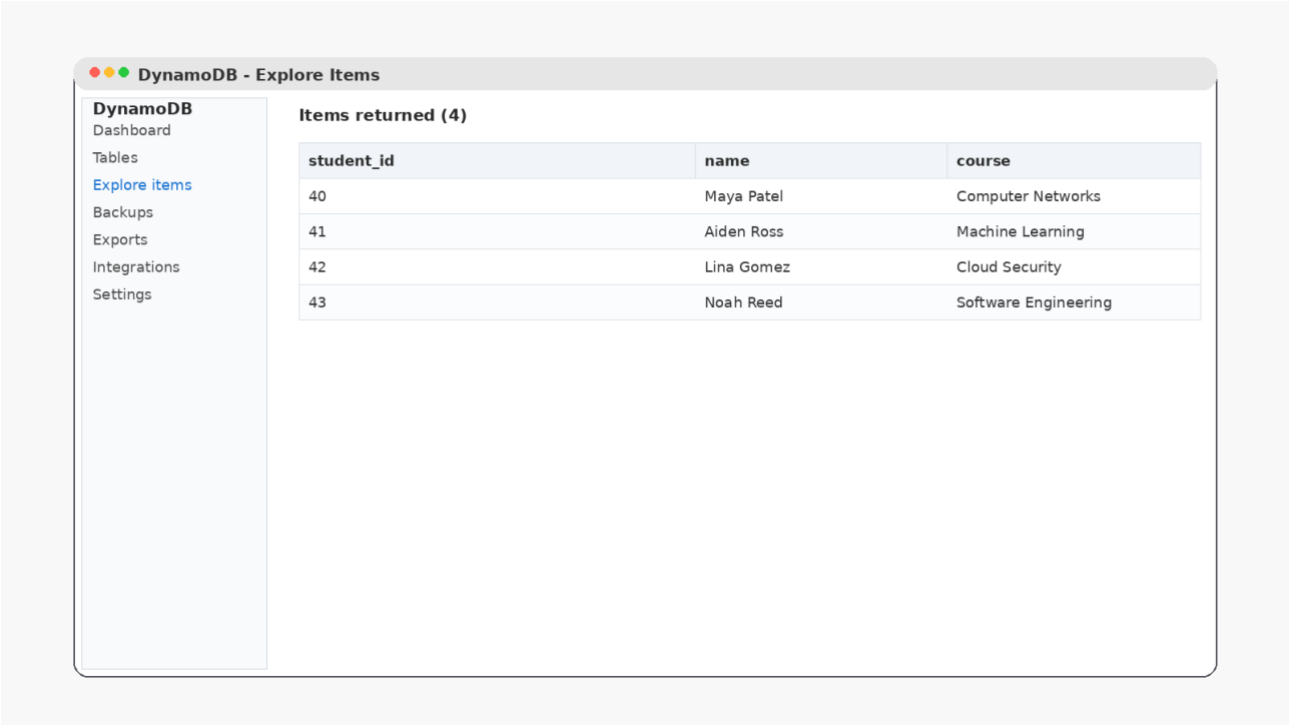


*Figure 6. Fabricated table snapshot after the update operation.*

| Operation | Endpoint | Input | Expected Result | Observed Status |
|---|---|---|---|---|
| POST | /students | new JSON record | insert new item | 200 OK |
| GET | /students?student_id=42 | existing key | return matching item | 200 OK |
| PUT | /students | updated course | modify existing item | 200 OK |
| DELETE | /students?student_id=42 | existing key | remove item | 200 OK |
| GET | /students?student_id=100 | missing key | error message | 404 Not Found |

## 5. Edge Cases, Challenges, and Learnings

| Scenario | Behavior | Resolution |
|---|---|---|
| GET on missing student_id | Handler returns 404 with a clear JSON message. | Checked whether Item exists in the lookup response before serializing output. |
| DELETE on missing student_id | Deletion is skipped and the client receives a user-facing error. | Added a pre-delete existence check to avoid silent success. |
| Unstructured event payload | Initial testing failed because httpMethod was missing. | Enabled proxy-style integration so the handler receives the full REST event. |

- Serverless APIs are convenient for coursework because they eliminate persistent server setup while still exposing realistic cloud integration patterns.
- DynamoDB favors access patterns around the partition key, so request design should align with the table schema from the start.
- Returning clear status codes for failure cases improves debugging and makes API behavior easier to validate in a report.